

Task 1: Memory Allocation Simulator.

```
#include <stdio.h>
```

```
int main() {
    int n;
    scanf("%d", &n);
    int part[2][n];

    for (int i = 0; i < n; ++i) {
        part[0][i] = 0;
        scanf("%d", &part[1][i]);
    }

    int m;
    scanf("%d", &m);
    int proc[3][m];

    for (int i = 0; i < m; ++i) {
        proc[0][i] = 0;
        scanf("%d", &proc[1][i]);
        proc[2][i] = -1;
    }

    int total_freq = 0;
    printf("\nFirst Fit\n");
    for (int i = 0; i < m; ++i) {
        if (proc[0][i] == 0) {
            for (int j = 0; j < n; ++j) {
                if (part[0][j] == 0 && part[1][j] >= proc[1][i]) {
                    total_freq += part[1][j] - proc[1][i];
                    proc[2][i] = j;
                    proc[0][i] = 1;
                    part[0][j] = 1;
                    break;
                }
            }
            if (proc[0][i] != 1) {
                proc[0][i] = -1;
            }
        }
    }

    printf("Allocation: ");
```

```

for (int i = 0; i < m; ++i) {
    printf("P%d -> ", i + 1);
    if (proc[0][i] > 0) {
        printf("%d ", proc[2][i]+1);
    } else {
        printf("Not Allocated ");
    }
}
printf("\n");
printf("Total Internal Fragmentation: %d\n", total_freg);

```

```

total_freg = 0;
for (int i = 0; i < n; ++i) {
    part[0][i] = 0;
}
for (int i = 0; i < m; ++i) {
    proc[0][i] = 0;
    proc[2][i] = -1;
}

```

```

printf("\nBest Fit\n");
for (int i = 0; i < m; ++i) {
    if (proc[0][i] == 0) {
        int best_fit = -1;
        int min_frag = 99999;
        for (int j = 0; j < n; ++j) {
            if (part[0][j] == 0 && part[1][j] >= proc[1][i]) {
                int frag = part[1][j] - proc[1][i];
                if (frag < min_frag) {
                    min_frag = frag;
                    best_fit = j;
                }
            }
        }
        if (best_fit != -1) {
            total_freg += min_frag;
            proc[2][i] = best_fit;
            proc[0][i] = 1;
            part[0][best_fit] = 1;
        } else {
            proc[0][i] = -1;
        }
    }
}
}

```

```

printf("Allocation: ");
for (int i = 0; i < m; ++i) {
    printf("P%d -> ", i + 1);
    if (proc[0][i] > 0) {
        printf("%d ", proc[2][i]+1);
    } else {
        printf("Not Allocated ");
    }
}
printf("\n");
printf("Total Internal Fragmentation: %d\n", total_freg);

```

```

total_freg = 0;
for (int i = 0; i < n; ++i) {
    part[0][i] = 0;
}
for (int i = 0; i < m; ++i) {
    proc[0][i] = 0;
    proc[2][i] = -1;
}

```

```

printf("\nWorst Fit\n");
for (int i = 0; i < m; ++i) {
    if (proc[0][i] == 0) {
        int worst_fit = -1;
        int max_frag = -9999;
        for (int j = 0; j < n; ++j) {
            if (part[0][j] == 0 && part[1][j] >= proc[1][i]) {
                int frag = part[1][j] - proc[1][i];
                if (frag > max_frag) {
                    max_frag = frag;
                    worst_fit = j;
                }
            }
        }
        if (worst_fit != -1) {
            total_freg += max_frag;
            proc[2][i] = worst_fit;
            proc[0][i] = 1;
            part[0][worst_fit] = 1;
        } else {
            proc[0][i] = -1;
        }
    }
}

```

```

    }
}

printf("Allocation: ");
for (int i = 0; i < m; ++i) {
    printf("P%d -> ", i + 1);
    if (proc[0][i] > 0) {
        printf("%d ", proc[2][i]+1);
    } else {
        printf("Not Allocated ");
    }
}
printf("\n");
printf("Total Internal Fragmentation: %d\n", total_freg);

return 0;
}

```

Input:

```

4
100 500 200 300
5
212 417 112 426 50

```

Output:

First Fit

Allocation: P1 -> 2 P2 -> Not Allocated P3 -> 3 P4 -> Not Allocated P5 -> 1
 Total Internal Fragmentation: 426

Best Fit

Allocation: P1 -> 4 P2 -> 2 P3 -> 3 P4 -> Not Allocated P5 -> 1
 Total Internal Fragmentation: 309

Worst Fit

Allocation: P1 -> 2 P2 -> Not Allocated P3 -> 4 P4 -> Not Allocated P5 -> 3
 Total Internal Fragmentation: 626